O'REILLY

# Hands-on Hugging Face for Natural Language Processing

# Prerequisites

- Comfortable programming in Python
- Basic familiarity with machine learning and deep learning
- Basic familiarity with natural language processing

# Poll Question

How comfortable are you building and training machine learning models in Python?

- Never worked with ML before
- Built traditional ML models using scikit-learn
- Built ML models using TensorFlow and PyTorch

# Poll Question

Have you worked with Natural Language Processing before?

- Never worked with NLP before
- Built traditional NLP models with scikit-learn
- Built NLP models with TensorFlow, PyTorch

# Hugging Face

A company known for its work in the field of artificial intelligence particularly in the field of natural language processing

# Hugging Face

A machine learning and data science platform and community that helps users build, deploy, and train machine learning models

# Hugging Face

Model Hub (huge repository of models)

Transformers (most influential contribution)

Datasets (ready to use datasets)

Spaces (ML demos and apps)

Inference (run APIs on their infrastructure)

# Focus of this Training

Transformers (most influential contribution)

# Natural Language Processing with Transformers

- The original transformer architecture was introduced in the 2017 paper “**Attention Is All You Need**”
- Introduced the novel concept of attention to focus on the right parts of the input
- Revolutionized the field of NLP, leading to the powerful LLMs that we work with today

# Hugging Face Transformer Library

- Offer a wide range of **pre-trained** transformer models for different use cases
- Cross-framework compatibility – all models can be used with both PyTorch and TensorFlow
- Used in a variety of applications – chatbots, enhancing search, translation, and more

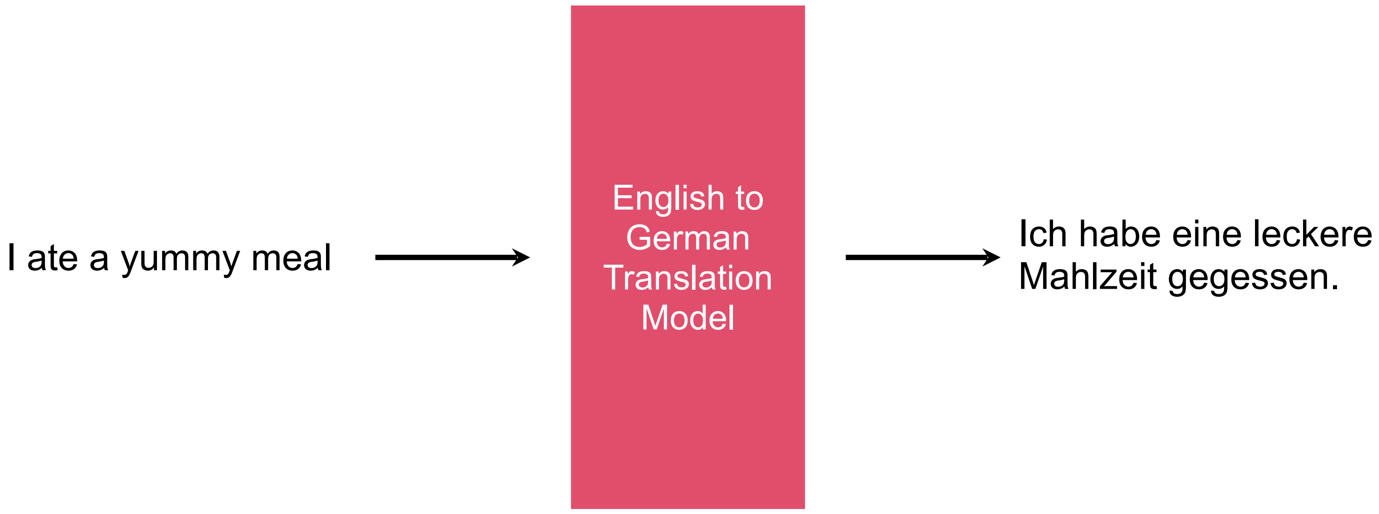
O'REILLY

# Attention and Transformer Models

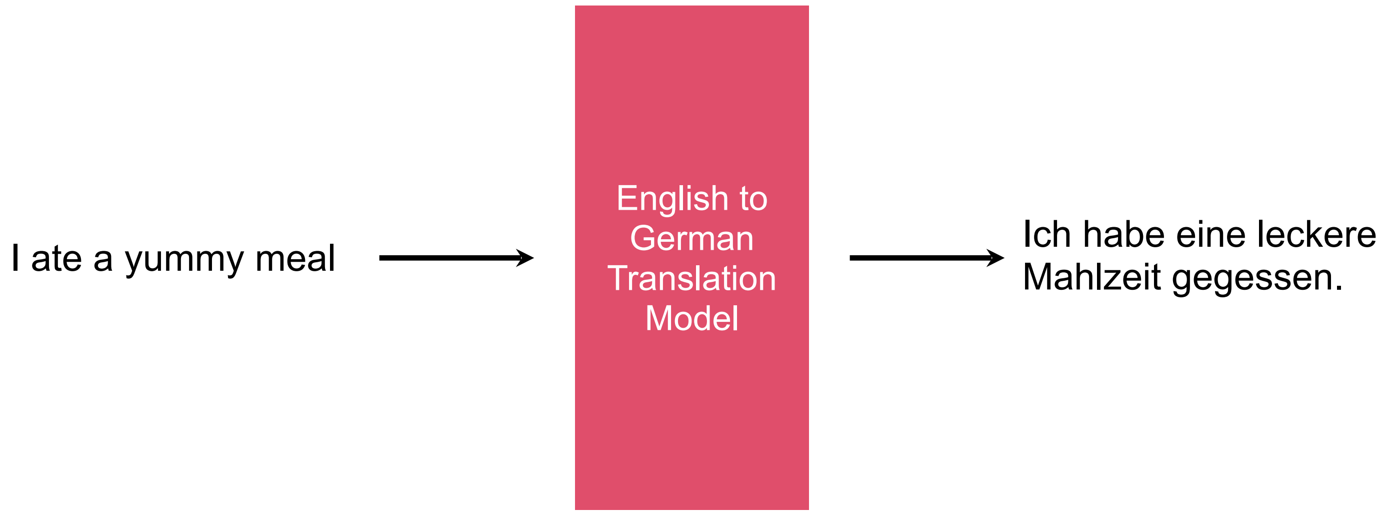
# Attention in Neural Networks

Mechanism that allows the network to focus on specific parts of the input data while ignoring others

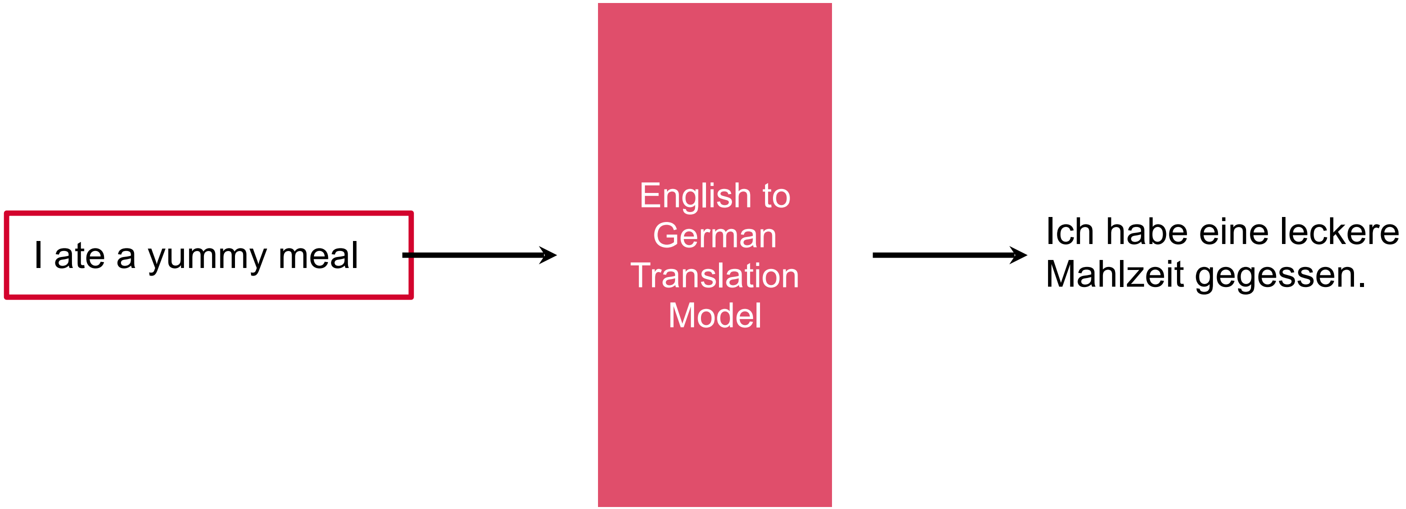
# Language Translation Model



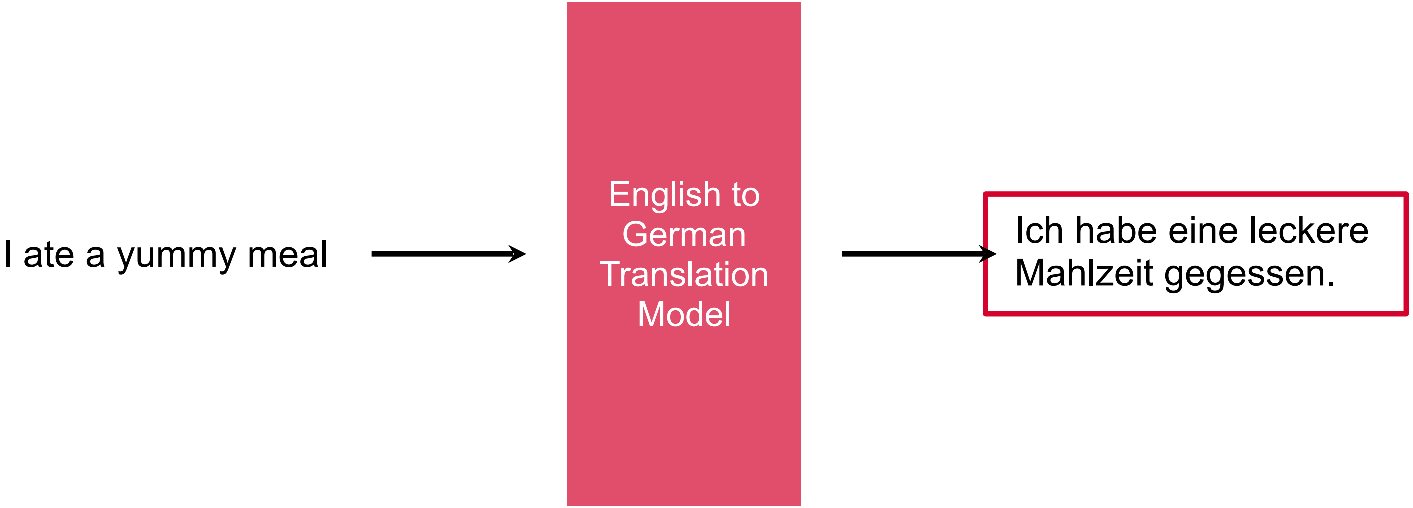
# Model Without Attention



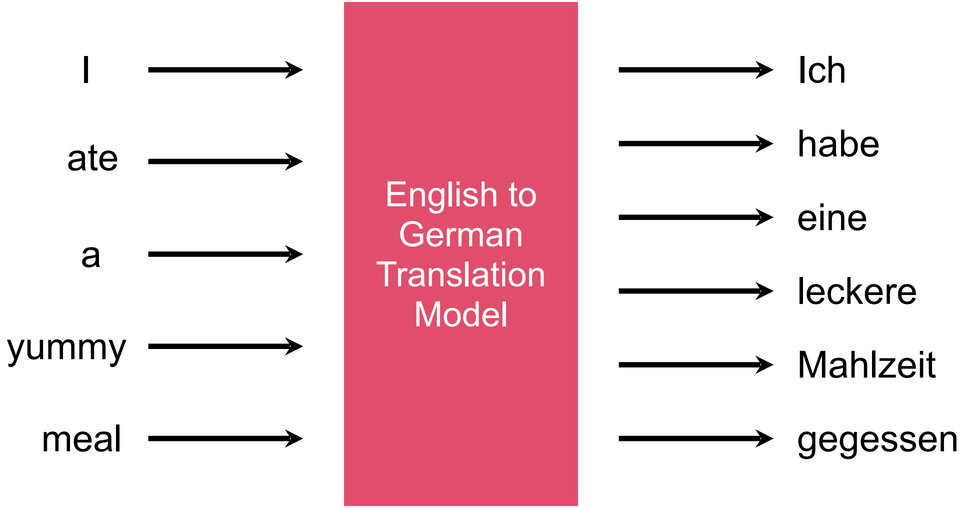
# Parses the Entire Input



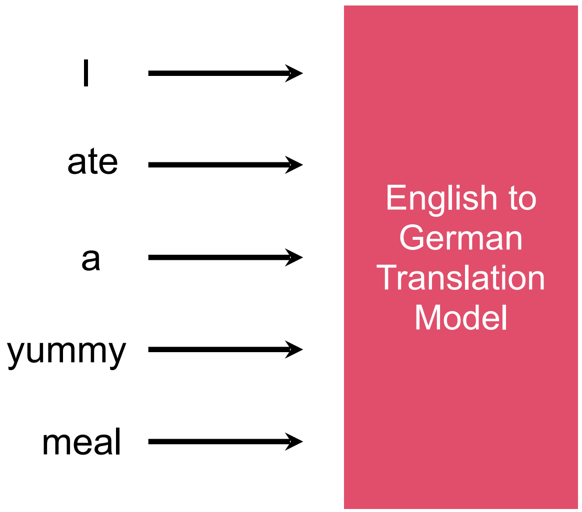
# To Generate the Translation



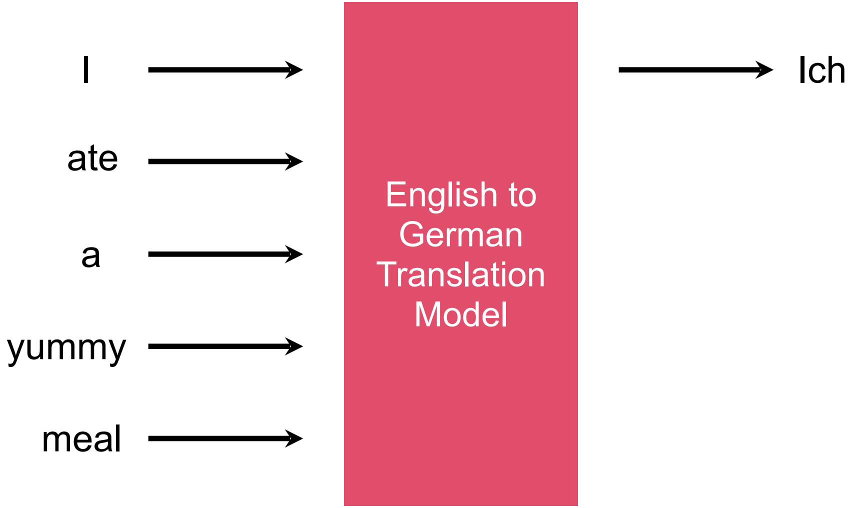
# Input and Output are Sequences



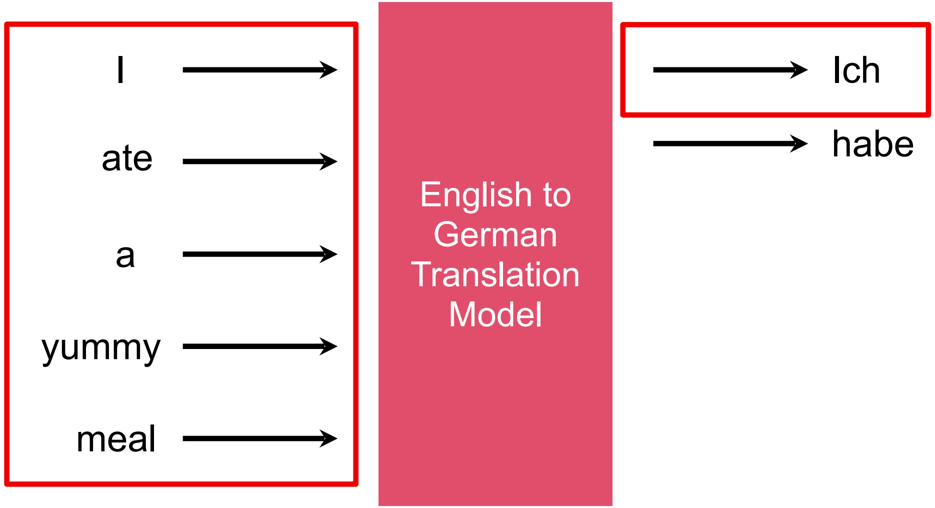
# Input Sequence Fed In



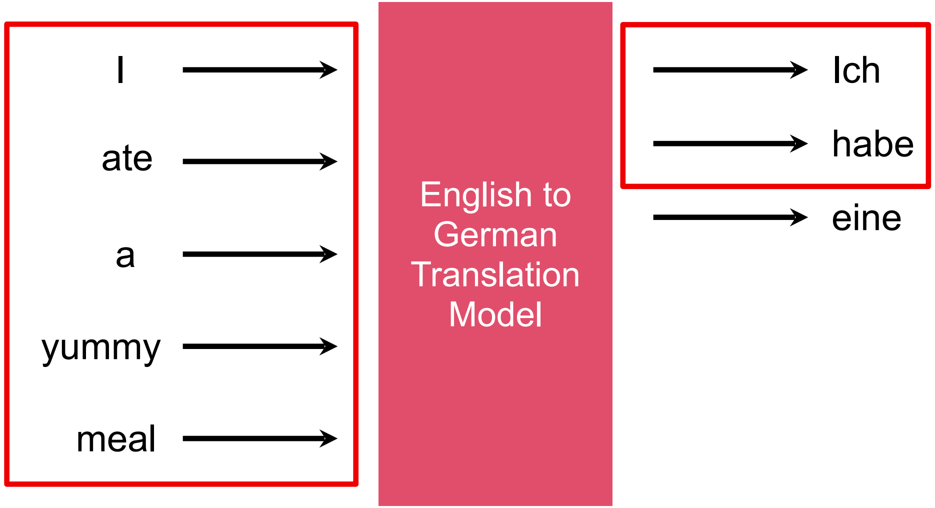
# Output Sequence Produced one Word at a Time



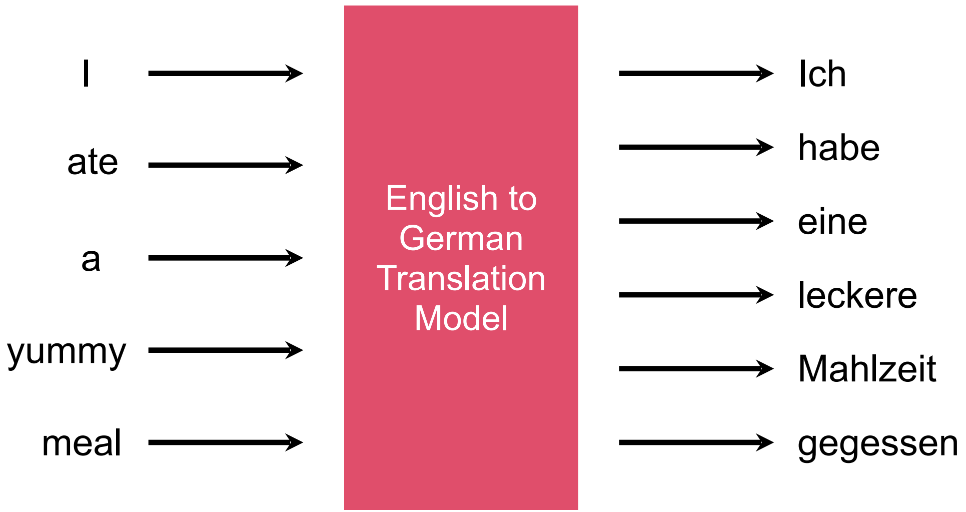
# Complete Input + Words Produces So Far Used to Get Next Word in Sequence



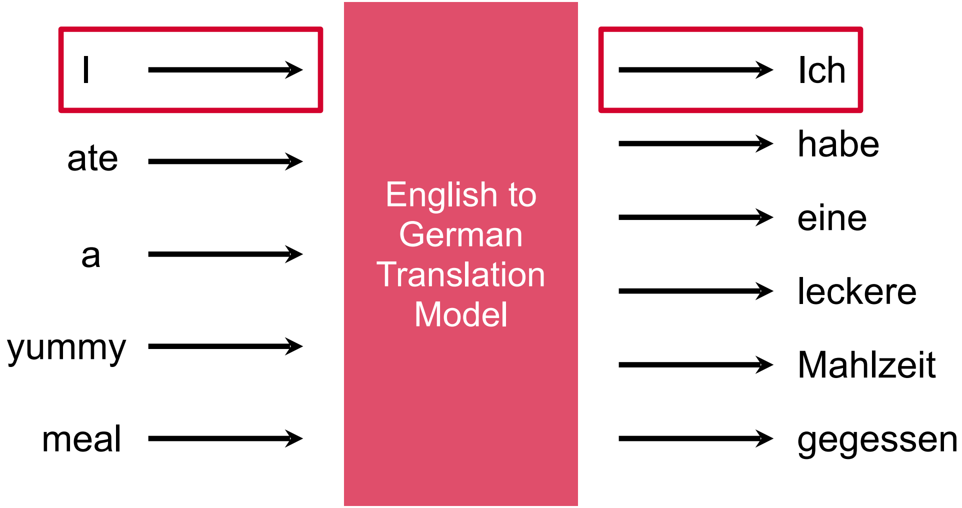
# Predict One Word at a Time



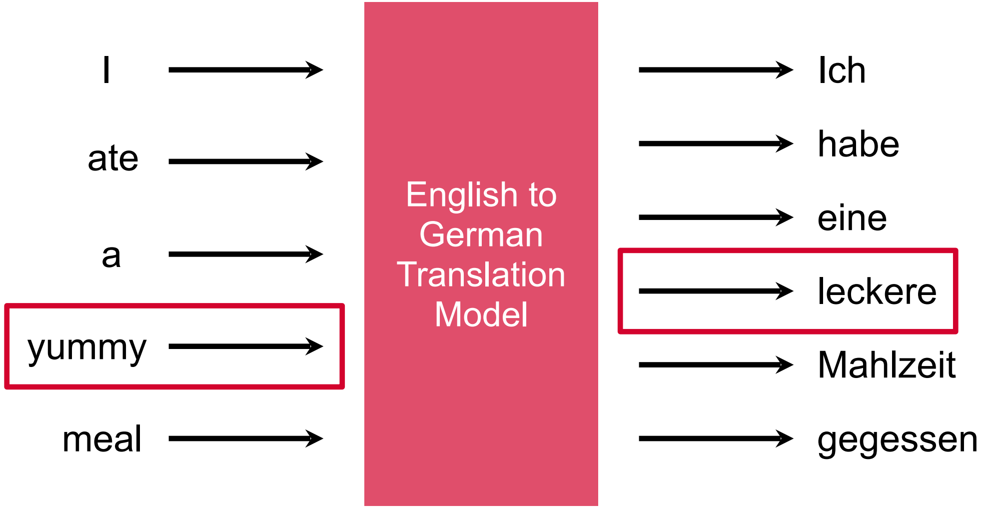
# Better Predictions with Attention



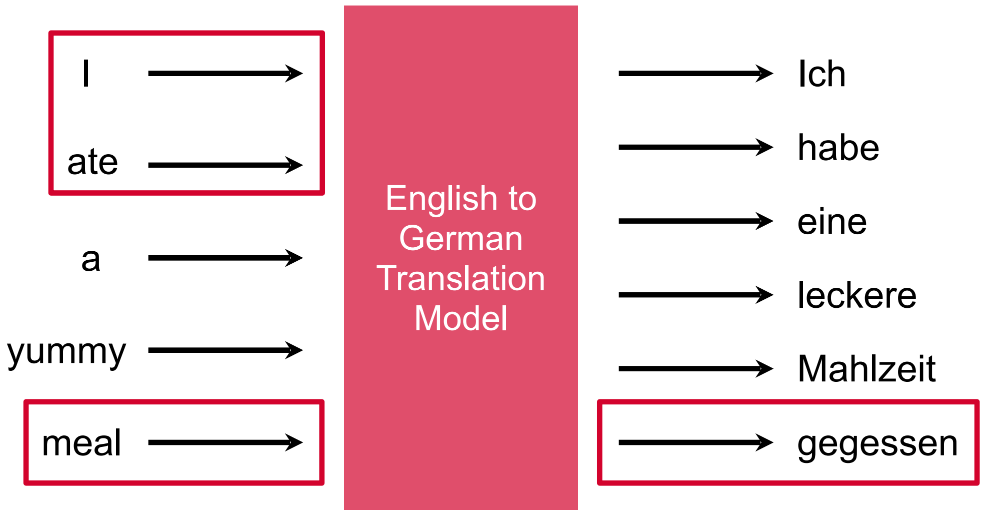
# Better Predictions with Attention



# Better Predictions with Attention



# Better Predictions with Attention



# Attention

- Help models focus on the right parts of the input to generate the output
- The entire input along with attention generates better outputs
- Are the foundation of transformers used to power the large language models of today

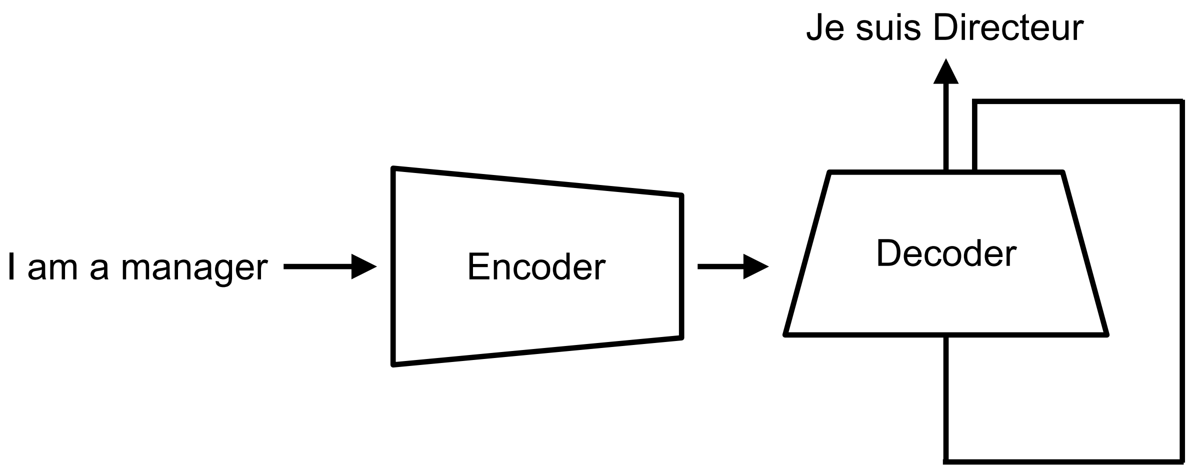
O'REILLY

# The Transformer Architecture

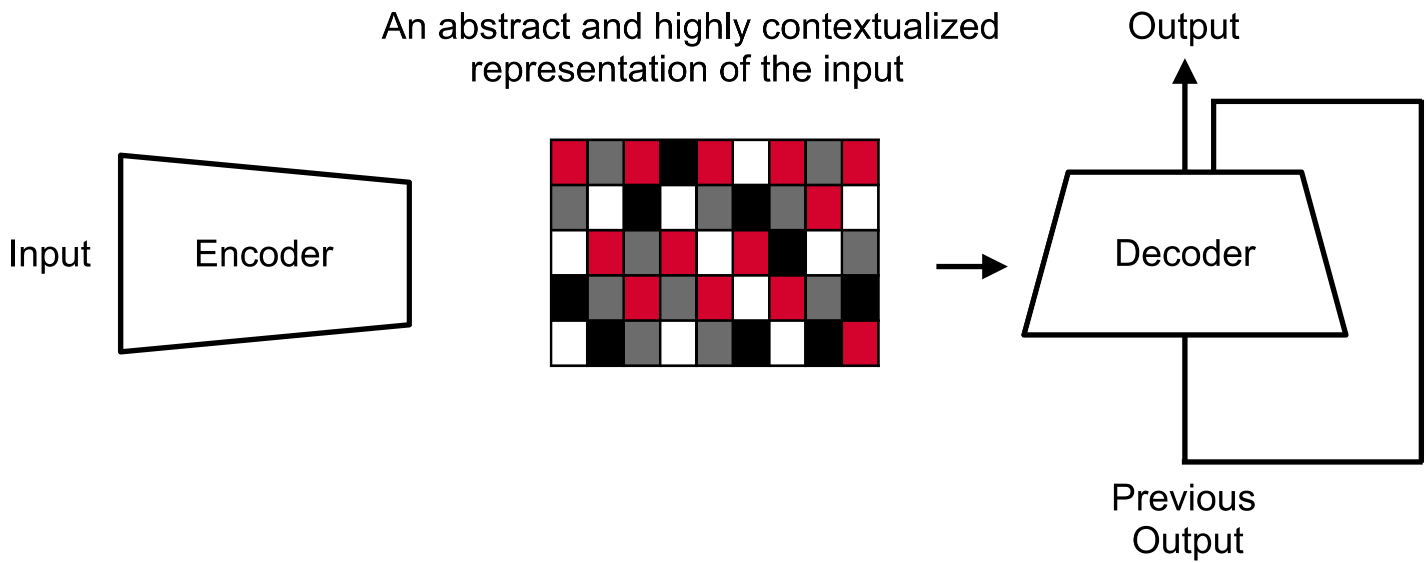
# The Transformer Architecture

A model architecture primarily used in natural language processing that relies on self-attention mechanisms to process sequential data more effectively than previous models

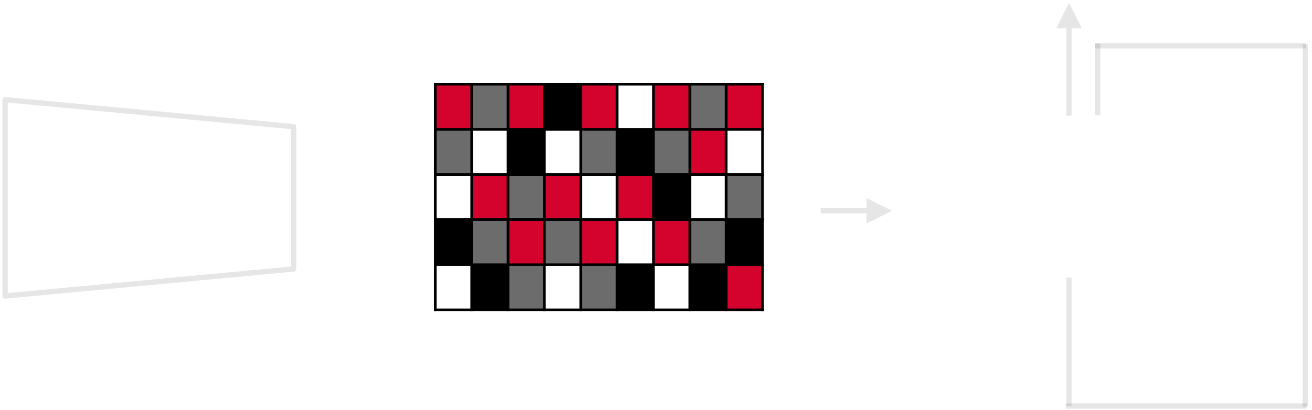
# High-level Architecture of Transformers



# Encoder Creates Abstract Representation Fed to the Decoder

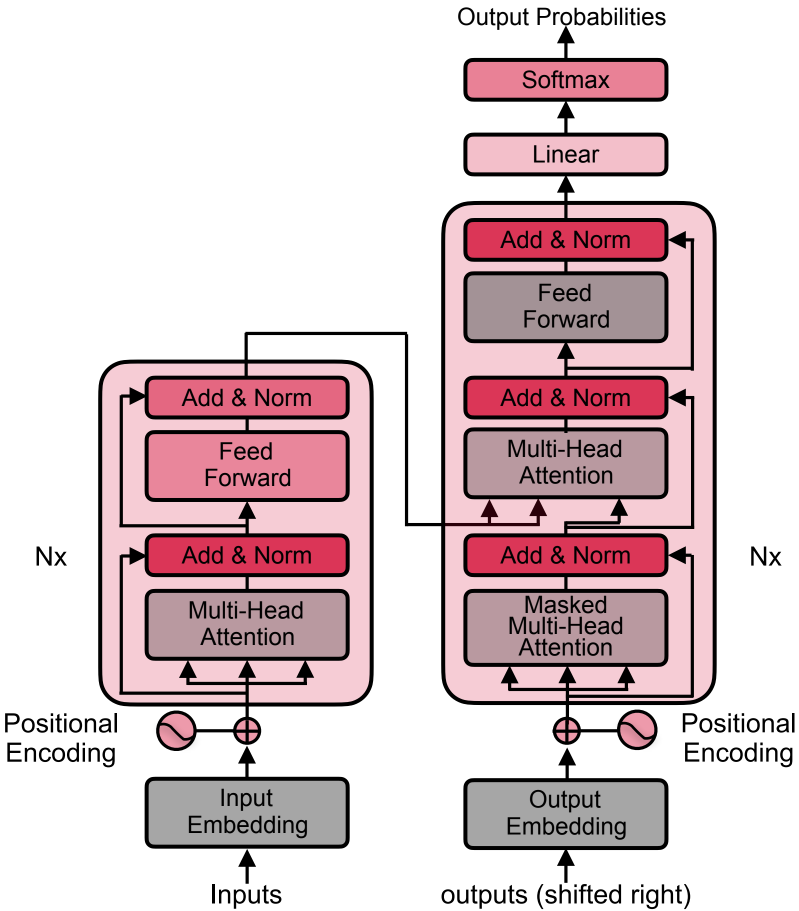


# Encoder Creates Abstract Representation Fed to the Decoder

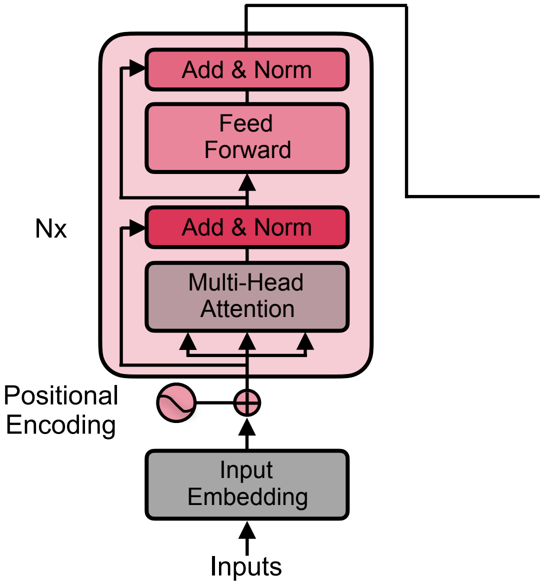


This representation encodes the importance of every word with respect to other words in the same input sequence

# Transformer Architecture

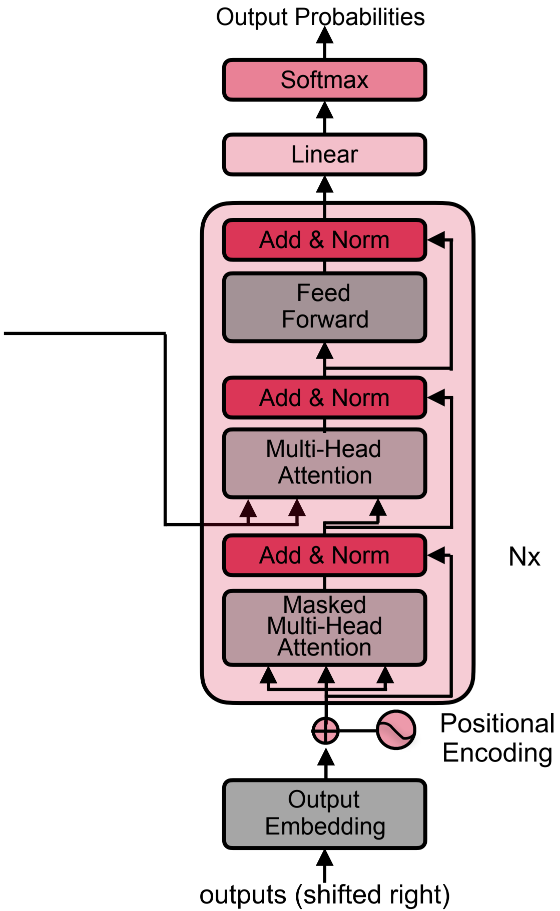


# Encoder



Add & Norm
Feed Forward
Nx
Add & Norm
Multi-Head Attention
Positional Encoding
Input Embedding
Inputs

# Decoder


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Positional Encoding
Output Embedding
outputs (shifted right)

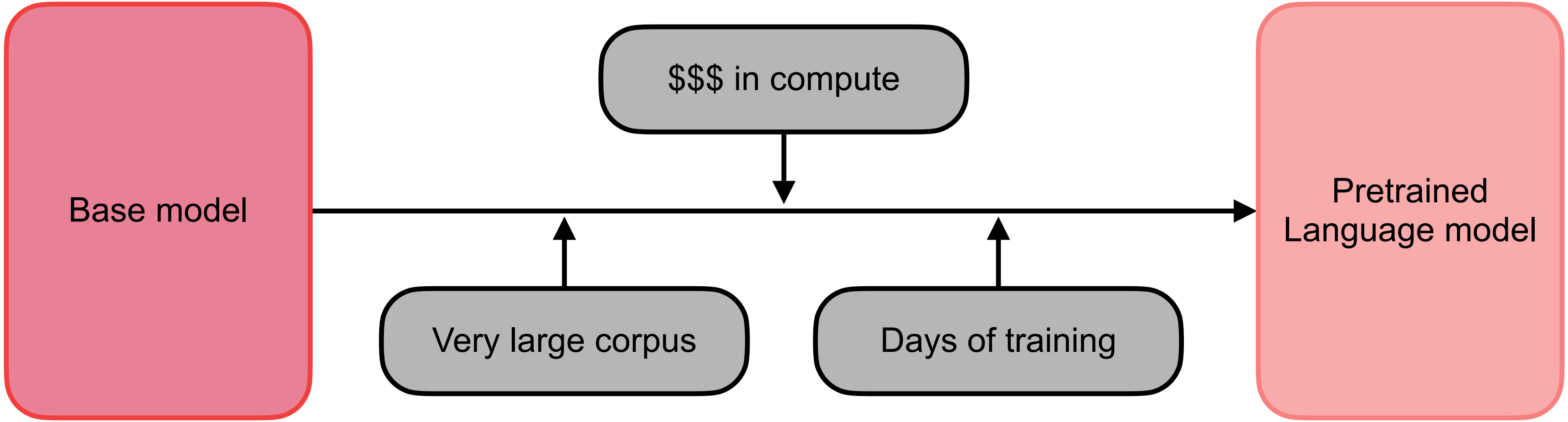
O'REILLY

# Transformers in Hugging Face

# Pretrained Transformers in Hugging Face

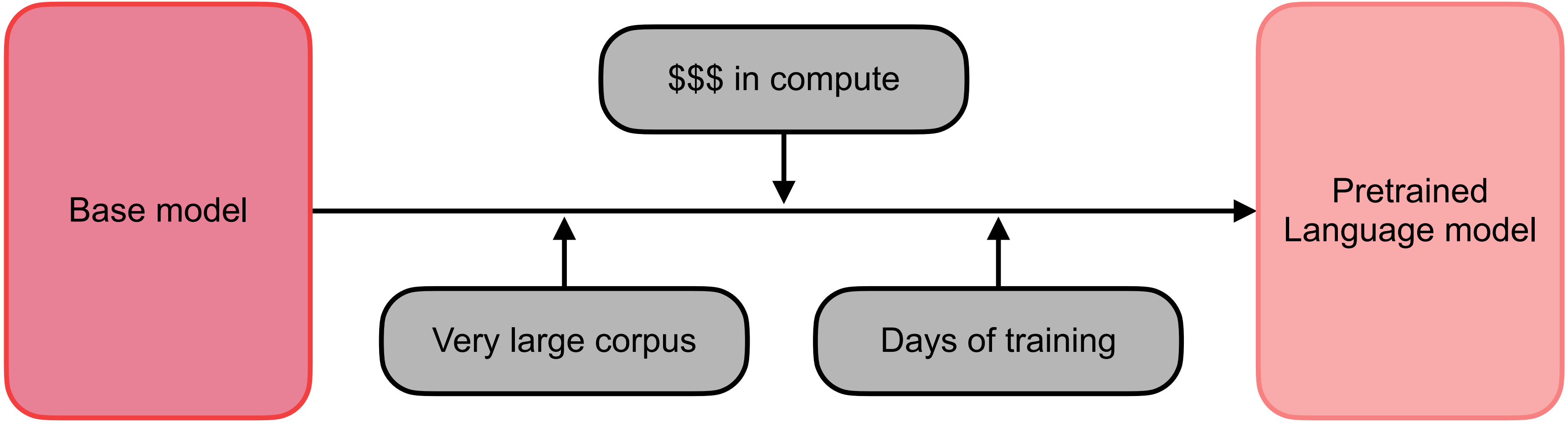
Transformers are very larger models and very expensive to train – in terms of data, time, and resources

# Training Transformers from Scratch



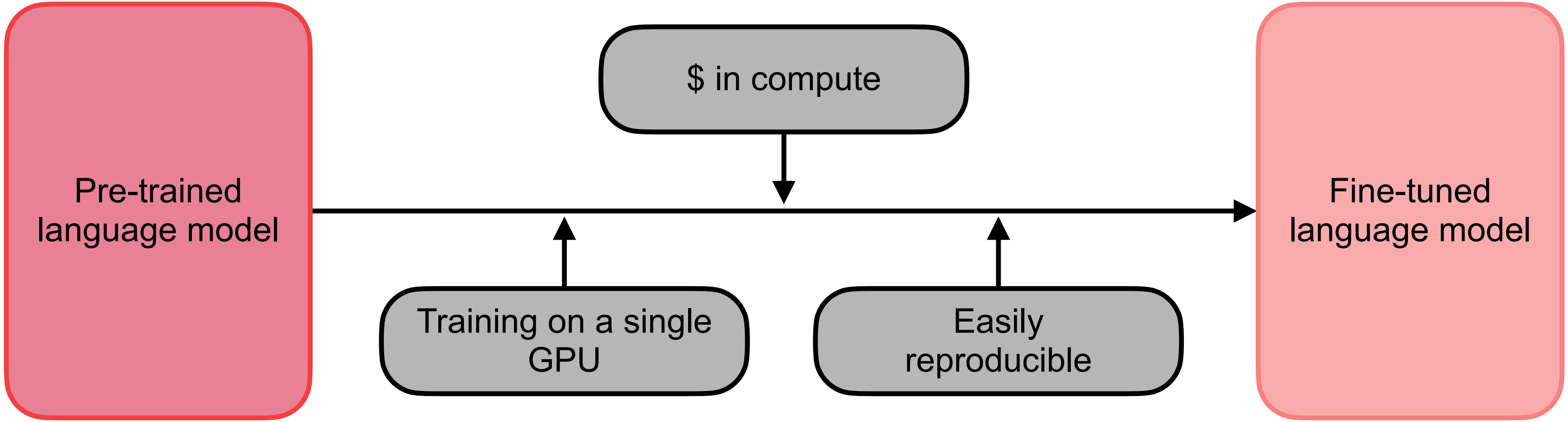

Training transformer models from scratch (pretraining) - model weights are initialized at random and the model has no prior knowledge embedded within it

# Training Transformers from Scratch



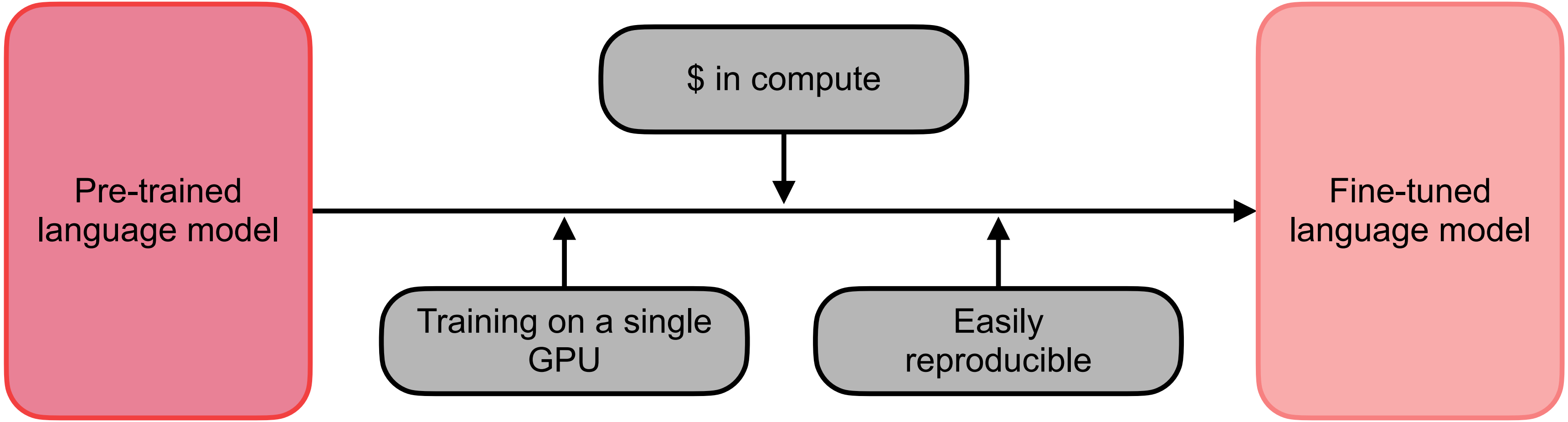

Needs huge amounts of data and compute resources - can be very expensive

# Fine-tuning Transformers




Fine-tuning models involve additional training on an already pretrained model

# Fine-tuning Transformers

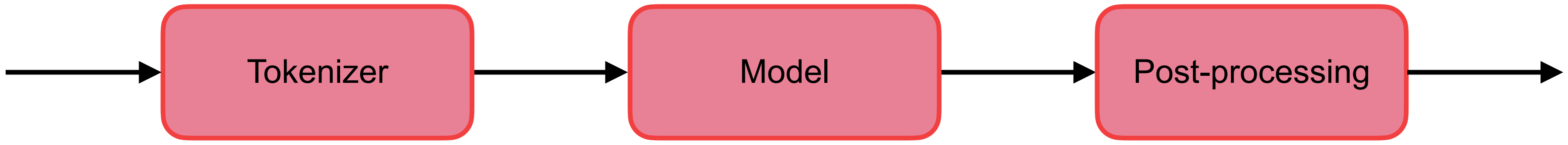



Use less data, time, and resources to get good results - overall costs and environment impact of training is much lower than training from scratch

# Hugging Face Pipelines

A high-level abstraction that makes it easy for users to perform NLP tasks using pre-trained models

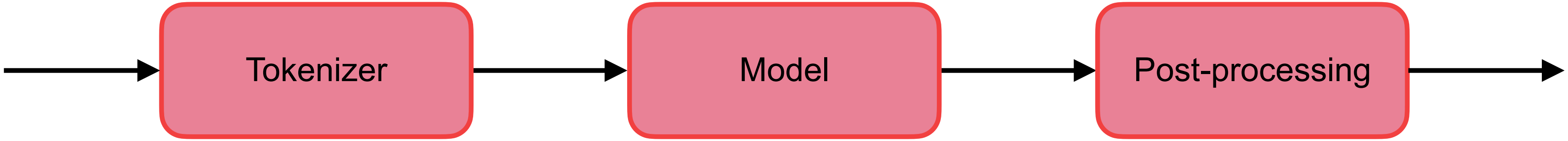
# Pipeline Components



O'REILLY

# Hugging Face Tokenizers

# Pipeline Components



# Tokenizer

# Tokenizing Text

Don’t you know what time it is?

# Split by Spaces

[“Don’t”, “you”, “know”, “what”, “time”, “it”, “is?”]

# Consider Punctuation

[“Don”, “‘”, “t”, “you”, “know”, “what”, “time”, “it”, “is”, “?”]

# Split Words Like Won’t and Don’t

[“Do”, “n’t”, “you”, “know”, “what”, “time”, “it”, “is”, “?”]

# Tokenizers

- Rule-based tokenizers with different rules split data in different ways
- Pretrained models only work well with **inputs tokenized in the same manner as its training data**

# Categories of Tokenizers

Word Tokenizers

Character Tokenizers

Subword Tokenizers

# Word and Character Tokenizers

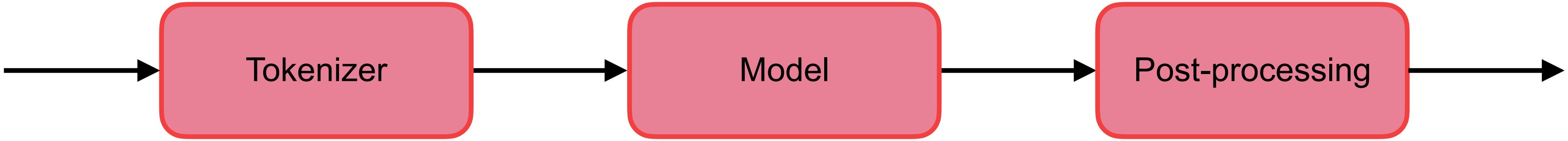
- Split text into tokens in meaningful ways
- “swim” and “swimming” should be identified as related
- Reduce the number of unknown words
- Word tokenizers
  - Too large a vocabulary
  - Will not identify related words
- Character tokenizers
  - Large number of tokens for each sentence
  - Intuitively less meaningful

# Subword Tokenization

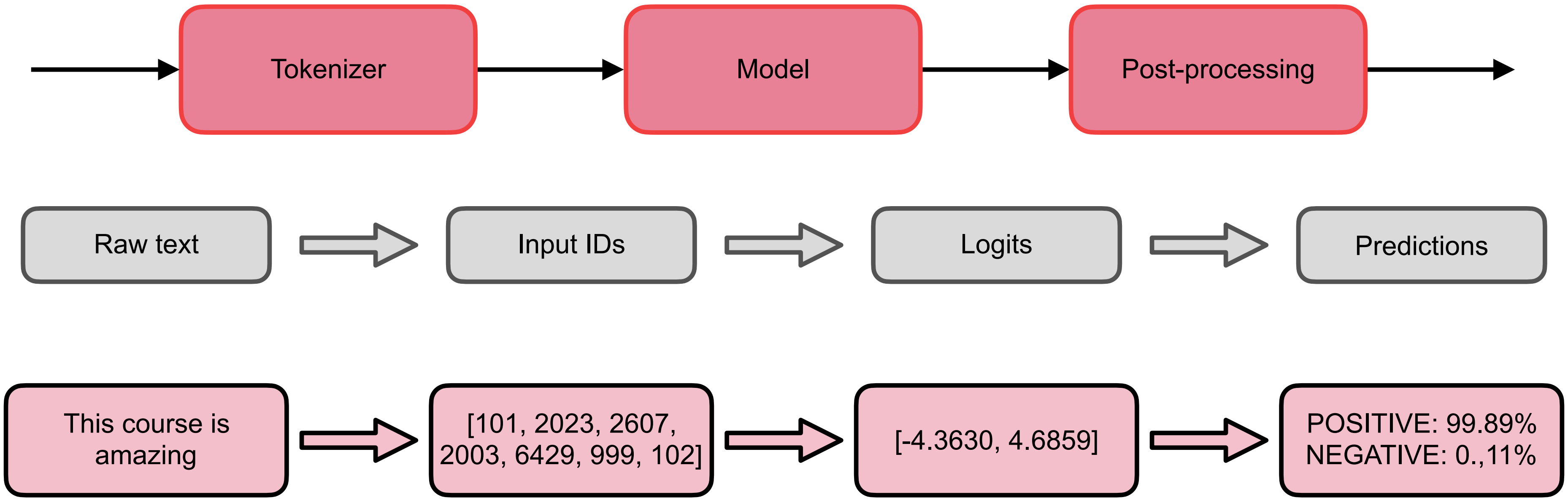
- Relies on the principle that frequently used words should not be split but rare words should be split into subwords
- “emotionally” could be a rare word, split into “emotional” and “ly”
- Allow models to have a reasonable vocabulary size and yet learn meaningful context independent representations

# Hugging Face Pipelines

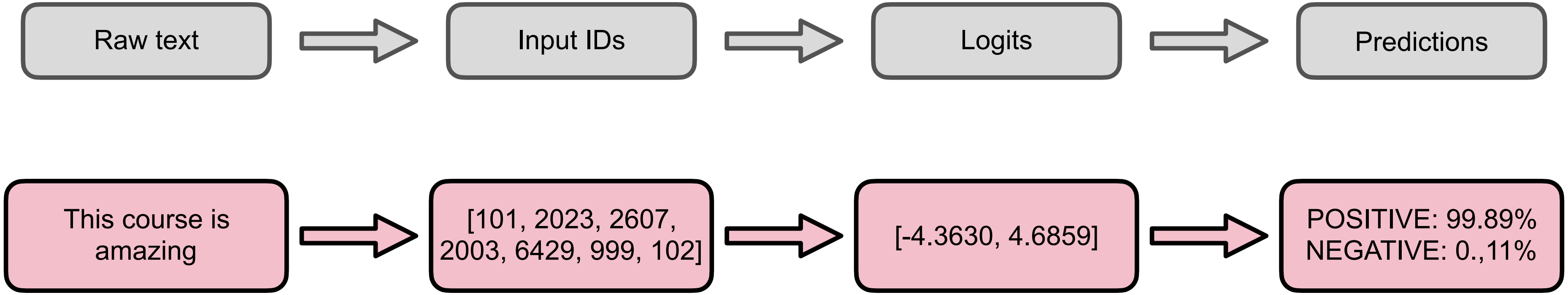
# Pipeline Components



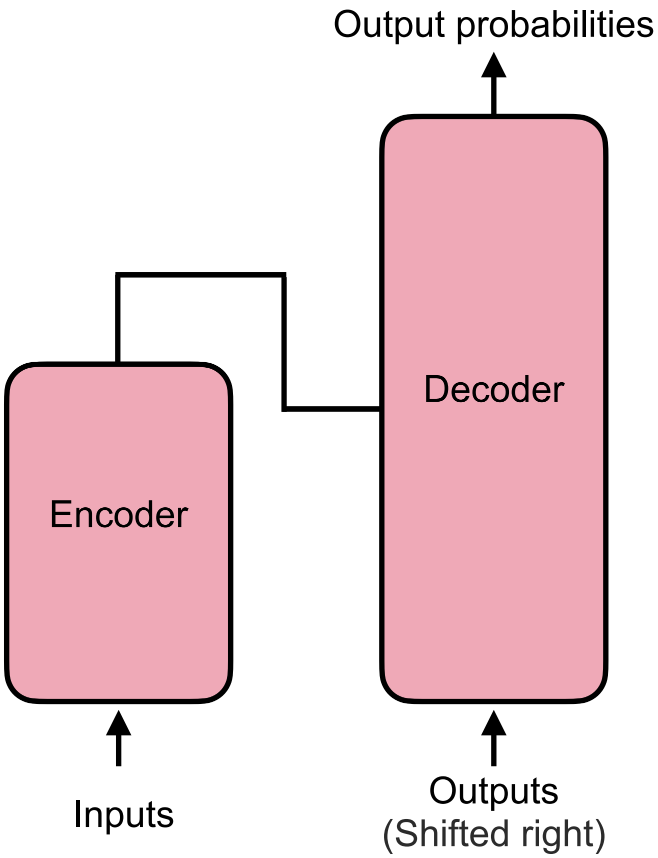
# Hugging Face Transformer Pipeline



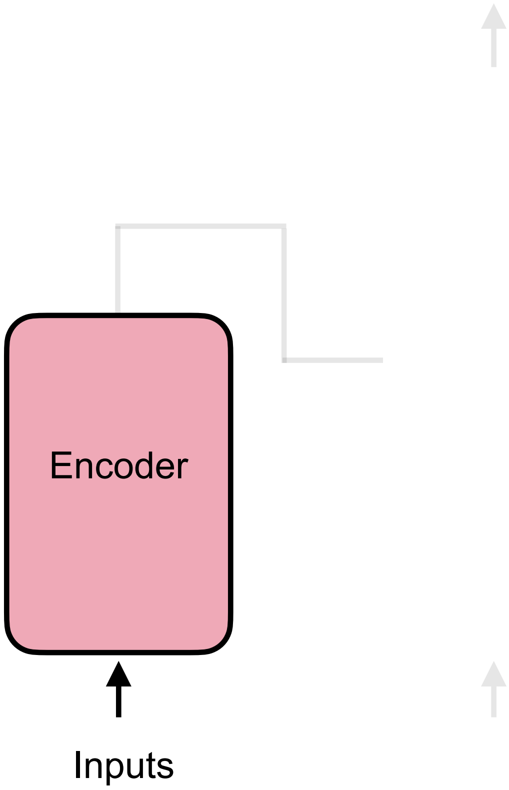
# Hugging Face Transformer Pipeline



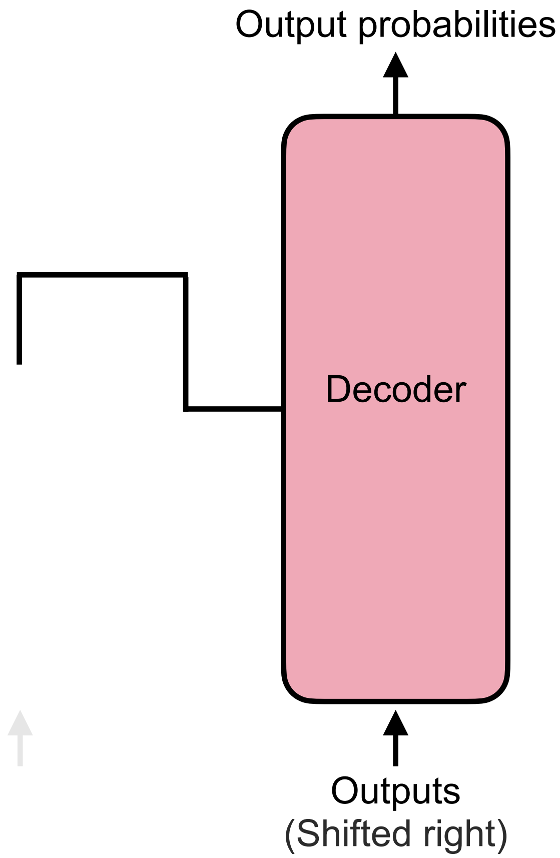
# General Transformer Architecture



# Encoder



# Decoder



# Encoder-only Models

- Attention layers in the encoder can access all the words in the input sentence called “bi-directional” or “auto-encoding” models
- Pre-training involves corrupting the initial sentence (masking) and having the model predict the masked word
- Suited for tasks such as **sentence classification, named entity recognition, and question answering**

# Masked Word Prediction

Let’s have some fun doing ______ programming!

Python?

Java?

C++?

# Decoder-only Models

- Attention layers in the decoder can only access words before the current word in the sentence – called “auto-regressive” models
- Pre-training of these models involve predicting the next word in the sentence, given all words seen so far
- Models are best suited for natural language tasks such as **text generation**

# Next Word Prediction

It’s a nice sunny afternoon. Let’s go _____

outside
swimming
running
dancing

# Encoder-Decoder Models

- Called sequence-to-sequence models
- Attention layers in the encoder can access all parts of the input sentence,
- Attention in the decoder can only access words positioned before the given word
- Pre-training usually done using the objectives of the encoder and decoder (masked word prediction and next word prediction)
- Suited for tasks such as **summarization, translation, and generative question answering**